Programming Project Report 5

Warren Roberts
100450116

Problem Statement:

A major goal of this project is to create realistic images by utilizing ray tracing algorithms. With this we can simulate reflection, refraction, and other lighting effects. Phong shaders worked side by side with our ray tracing logic, allowing for a robust and beautiful environment. Another goal of this project was to be able to reproduce a specific scene based on requirements. These are important topics in computer graphics and extended my knowledge greatly. The things I learned on this course will greatly help me along the road of becoming a game developer, so thank you John Gauch! Normal inputs for this program include Q and E to zoom in / out on the scene. This allows users to get a closer or further look at the scene, giving a dynamic point of view to the user. The program should output a full scene replicating the Turner Whitted's Sphere Animation. Alongside this, there is another object (the Cube model), that is rendered in the scene. I also made them rotate to demonstrate the next feature well and dynamically. The next feature allows for casting reflection rays (like a mirror), that reflect other objects in the scene. All these features should be visible when the program is run without zooming. Division by zero in many cases was prevented to avoid errors, as well as preventing self-intersection with small offsets on each ray. Invalid intersection distances were also checked to ensure correct outputs.

Design:

Like the other projects I made sure to follow a strict OOP standard. I started by copying project 4's work over to a new homework 5 folder. I kept model, controller, app, view, and core functionality in their own respective folders. To enforce polymorphism, I used unique pointers to work as a vector of objects in the scene. I also used many data structures defined in the given ray tracer code, such as Ray3D, ColorRGB, Vector3D, and Point3D. In extension to these I used my own object classes for the checkered plane, cubes, and spheres. The primary algorithm used was ray tracing and was provided. This code was adapted to fit into my OOP style project setup and worked well. Algorithms were also made for cube, spheres, and plane intersections. One advantage is that the flow of the program was very easy to follow, polymorphism makes the setup of objects much easier as well! Using unique_ptr also allowed safe and efficient memory management of objects. A limiting factor would be that cube intersection assumes axis aligned geometry, meaning that object rotations do not affect the ray tracing unless transformations are applied. This took extra work and added to the complexity of the cube objects functionality (to make it rotate on its axis).

Implementation:

I began by understanding the provided ray tracing pipeline and making sure I could compile and run the base program correctly. From there, I focused on how rays are

generated per pixel and how the closest object intersection is determined. I started with John Gauch's source code for ray tracing, as well as all code from my implementation of project 4. I adapted this code by splitting the ray_trace.cpp and .h, dividing the work they do in an OOP correct form. Allocating object work to objects and cameras through our camera and controller files. This project was completed from April 22nd to April 30th to complete. I spent the first couple days translating the ray trace file into OOP, then a couple days implementing step one. After this the rest of the time was spent implementing feature 2 and 3and debugging.

AI Interaction Log

Entry 1 – RayTracer Design + OOP Structure
Prompt:
Explain all concepts in a ray tracer file and determine how to split it into multiple files while maintaining OOP principles.
Summary of Response:

- **Broke down core ray tracing pipeline:**
  - **camera setup**
  - **ray generation**
  - **object intersection**
  - **normal calculation**
  - **shading (Phong)**
- **Identified responsibilities inside RayTracer**
- **Recommended separation into:**
  - **RayTracer (rendering control)**
  - **Scene (object container)**
  - **Object3D (base class)**
  - **Sphere3D, etc.**
  - **Material, Light, HitInfo**
- **Emphasized separation of concerns and scalability**

Impact:
Led to a cleaner architecture and improved maintainability of the rendering system.

Entry 2 – Object Placement (OOP Design Decision)
Prompt:
Should objects be defined inside raytracer.cpp or in separate files?
Summary of Response:

- **Recommended placing objects (e.g., Sphere3D) in their own files**
- **Kept RayTracer focused on rendering logic only**
- **Explained improved readability, tracing, and scalability**

Impact:
Improved code organization and made it easier to expand the scene with new objects.

Entry 3 – Checkerboard Ground Implementation
Prompt:
How to create a checkered ground surface?

Summary of Response:

- **Introduced Plane3D object**
- **Explained ray-plane intersection math**
- **Showed checker pattern using:**
  - **floor(p.x * scale)**
  - **floor(p.z * scale)**
- **Emphasized using correct axes (XZ for ground)**
- **Highlighted need for closest-object comparison**

Impact:

Enabled procedural ground texturing and added visual complexity to the scene.

Entry 4 – Cube Ray Tracing Implementation

Prompt:

How to convert a cube object into a ray-traced object?

Summary of Response:

- **Explained ray-box intersection (slab method)**
- **Introduced axis-aligned bounding box (AABB)**
- **Provided approach for:**
  - **computing intersection t**
  - **determining surface normals**
- **Recommended returning closest hit**

Impact:

Allowed rendering of cube objects using ray tracing instead of rasterization.

Entry 5 – Checkerboard Distortion Debugging

Prompt:

Why do checkerboard squares stretch and converge?

Summary of Response:

- **Explained perspective projection (expected behavior)**
- **Identified actual distortion causes:**
  - **missing aspect ratio correction**
  - **unnormalized ray directions**
  - **incorrect axis usage**
- **Provided fixes**

Impact:

Resolved visual distortion issues and improved correctness of rendering math.

Entry 6 – Background Color Modification

Prompt:

How to change background from black to sky blue?

Summary of Response:

- **Explained background as default pixel color**
- **Recommended setting pixel color before intersection tests**
- **Provided optional gradient sky implementation**

Impact:

Improved scene appearance and visual realism.

Entry 7 – Performance / Lag Issue
Prompt:
Is lag normal on Mac M1 or should optimization be done?
Summary of Response:

- **Identified main issue as pixel count, not architecture**
- **Recommended:**
  - **lowering resolution**
  - **using delta time for animation**
  - **avoiding unnecessary re-rendering**

Impact:
Improved runtime performance and reduced animation stutter.

Entry 8 – Resolution Optimization Decision
Prompt:
Lower resolution fixed lag—what should be done next?
Summary of Response:

- **Confirmed resolution scaling as expected performance bottleneck**
- **Recommended:**
  - **using lower resolution for real-time rendering**
  - **higher resolution only for final output**

Impact:
Balanced performance with visual quality efficiently.

Entry 9 – Reflection Realism Improvements
Prompt:
How to make reflections less glass-like and more realistic?
Summary of Response:

- **Introduced material-based reflectivity**
- **Provided blending formula:**
  - **final = local*(1 - reflectivity) + reflected*reflectivity**
- **Added:**
  - **recursion depth limits**
  - **reflection ray offset**
  - **realistic reflectivity ranges**
  - **specular adjustments**

Impact:
Improved realism of reflective surfaces and prevented over-reflective artifacts.

Entry 10 – Compilation Error Debugging
Prompt:
Fix build errors involving mPoint3D and reference issues
Summary of Response:

- **Identified typo: mPoint3D → Point3D**
- **Explained cascading compiler errors caused by incorrect type**

Impact:
Resolved compilation issue quickly and restored build functionality.

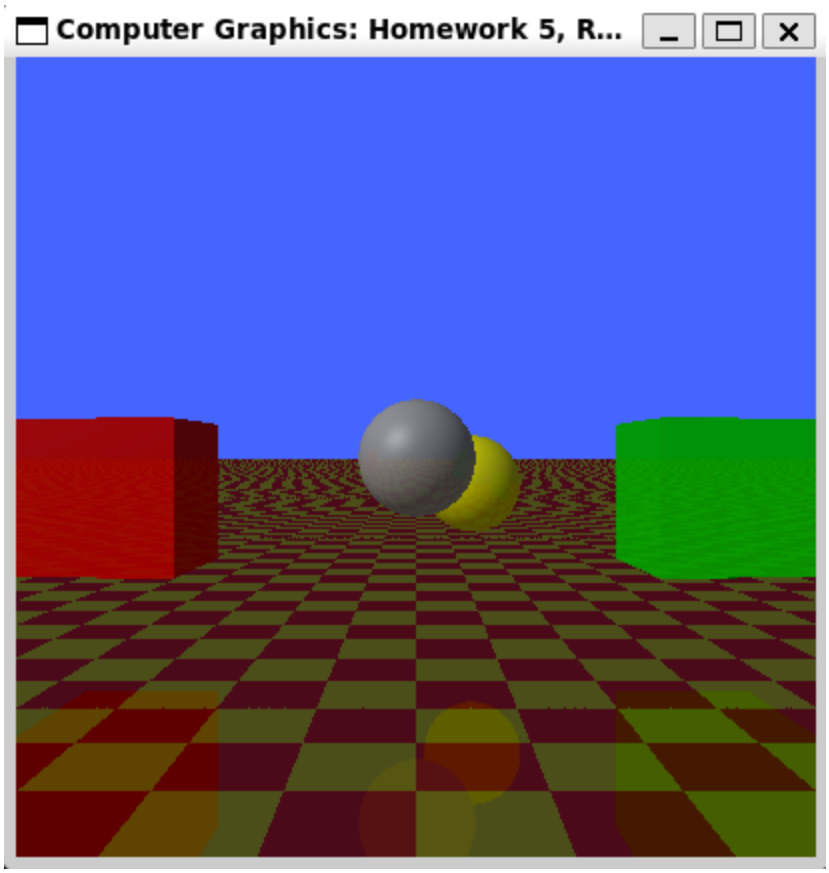
Conclusion
AI assistance was used as a development aid to:

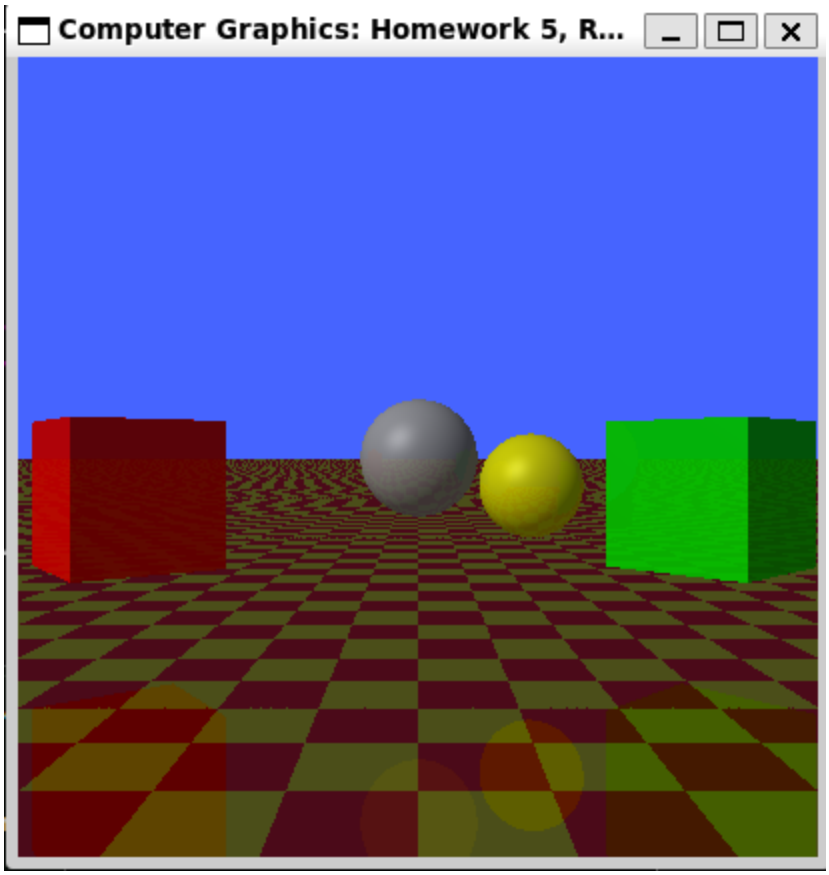
- **accelerate debugging,**
- **clarify complex graphics concepts,**
- **guide architectural decisions,**
- **and improve rendering realism.**

All final implementations were integrated, tested, and verified independently.

Testing:

I tested my program by constantly running it and checking each change, to do so a make file made my life a lot easier. I made colors extremely obvious and made the window size small to improve performance when testing. I had to test a lot of rendering issues as well by making small changes at a time and running it up. For normal inputs we had rays cast from a camera position toward the image plane, a directional light source, and valid surface normals and material values. To show rays that miss all objects was a special input as it involved creating and rendering a sky background everywhere without an object. Very close intersections also were a special input, and I had to add an offset. Everything worked as expected after much debugging. Basic ray tracing, animations, shading, shadow, and reflections are all simulated in this project. Reflection was limited though, due to only being able to bounce once (not realistic).


Computer Graphics: Homework 5, R...



Conclusions:

Overall, this project went very well. I was quite behind, but still able to complete it all. Using AI was absolutely a good helper to bridge the gap of misunderstood concepts and to hasten the development process. The project was a success, and I was able to implement feature 1-3 as required and I think everything looks pretty good. I did learn that I am not a fan of ray tracing though and will probably not be using this setup often. If I were to do this again, I would give myself more time, as the crunch made it stressful, but it would also be nice to implement the 4th feature and see how everything looks together! This project took 8 days to complete between April 22nd and April 30th, with a good couple of hours most days invested in development.